

ESE 1900 Lab Handbook

By Students for Students

Draft of February 18th

Drafted by Dimitris Deliakidis

Contents

1	Course logistics	3
1.1	Class Structure + Grading	3
1.2	Communications	3
1.3	How to do well in class	3
1.4	Office Hours	3
2	Lab 0 Introduction to Arduino	5
2.1	What is an Arduino	5
2.2	Downloading and Using Arduino IDE	5
2.2.1	Downloading Arduino IDE	5
2.2.2	Using Arduino IDE	6
2.3	What is a breadboard	6
2.4	Components you will need for this lab	7
2.5	Lab 0.1: Light up an LED without code	7
2.6	Lab 0.2: Light up an LED with code	8
2.7	Lab 0.3: Turn on an LED using a pushbutton	11
2.8	Lab 0 deliverables	12
3	Lab 2: Sensing	13
3.1	Lets start with some theory first	13
3.2	Lab 2.1: Ping Sensor Serial Monitor readings	14
3.2.1	How to wire the ping sensor to the arduino.	15
3.2.2	Code for Lab 2.1	16
3.3	Lab 2.2: Ping Sensor controlling RGB LED	17
3.3.1	Wiring is simple.	17
3.3.2	Code for Lab 2.2	18
3.4	Lab 2.3: Temp/Humidity sensor readings in Serial monitor	20
3.5	Lab 2 deliverables	21

1 Course logistics

ESE 1900 is designed for non-ESE majors interested in learning about hardware and building projects. The course includes weekly labs, assignments, and a final project.

1.1 Class Structure + Grading

Before each lab session, a pre-lab assignment must be completed and submitted at the beginning of the lab. The post-lab assignment is due at the end of the following lab session.

Lab Attendance	Lab Assignments	Final Project
10%	50%	40%
Attending lab on time every week Attending final project lab sessions	Pre Labs, Post Labs, any other lab-specific deliverables	Ideas Selection, MVP, Demo Day, Video, Meetings with Prof. and TAs etc.

1.2 Communications

Join ESE 1900 Slack Channel. We use it for:

- Course announcements
- Access to TAs for questions
- Advance absence notice
- Final project group communication

1.3 How to do well in class

- Ask questions! Being curious is a big plus for the class
- Submit your assignments on time
- Go to labs: You don't have an option
- Final project group communication

1.4 Office Hours

You are welcome to attend office hours if you have any questions about the lab or would like to elaborate on any of its contents. Office hours are also available for make-up labs. Feel free to stop by for an engaging discussion about your final project, including ideas on how to improve it or troubleshoot any issues. The table below provides you with the available office hours of the spring semester 2025

Names	Time/Day	Location
James/Dea	Tuesday 1:30 PM - 3:30 PM	Detkin
Dea	Monday 12:30 PM - 2:30 PM	Detkin
Cindy, Aadam, Kunal	Monday 3:00 PM - 5:30 PM	Detkin
Sherry	Friday 8:00 PM - 10:00 PM	Detkin
Dimitris	Thursday 5:30 PM - 7:30 PM	Detkin
Arnuv, Andy	Friday 3:30 PM - 5:30 PM	Detkin

Table 1: *Office Hours Schedule*

2 Lab 0 Introduction to Arduino

In your first lab you will configure your arduino to blink an LED and control it with a pushbutton

2.1 What is an Arduino

The Arduino Uno Board is one of many microcontroller development boards for hardware hacking and prototyping. Arduino facilitates interaction between the physical world and the “information world” using sensors, actuators, and other electronics, the behavior of which you control with code. Usually powered using a USB Cable and can supply up to 5 volts. **Think of it as a very small computer which can be programmed to interact with electronics such as sensors, motors, and lcds.**

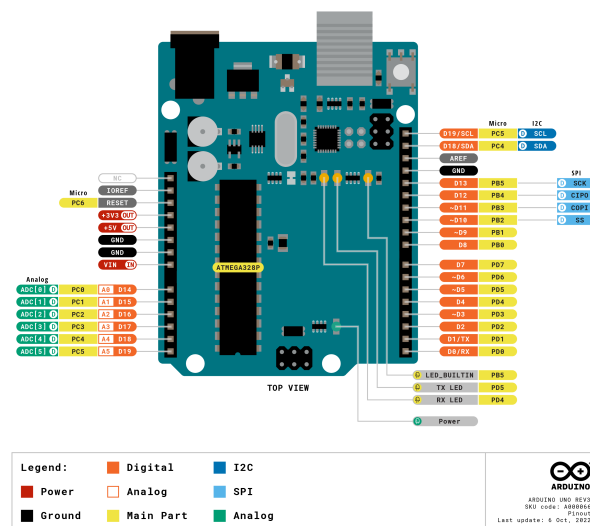


Figure 1: PinOut Diagram

2.2 Downloading and Using Arduino IDE

To work with Arduino, you need to install the Arduino Integrated Development Environment (IDE). Follow the steps below to download and use it:

2.2.1 Downloading Arduino IDE

1. Go to the official Arduino website: <https://www.arduino.cc/en/software>
2. Choose the appropriate version for your operating system (Windows, macOS, or Linux).
3. Download the installer and run it.
4. Follow the on-screen instructions to complete the installation.

2.2.2 Using Arduino IDE

1. Open the Arduino IDE after installation.
2. Connect your Arduino board to your computer using a USB cable.
3. Select your board model by navigating to: Tools → Board → Arduino Uno (or your specific board).
4. Select the correct port by going to: Tools → Port → (Select the appropriate port).
5. Write or open an existing Arduino sketch (program).
6. Click the **Upload** button to upload the code to your Arduino.
7. Observe the LED behavior or output as per your program.

For additional help, refer to the official Arduino documentation: <https://www.arduino.cc/en/Guide/HomePage>

2.3 What is a breadboard

A Breadboard is simply a board for prototyping or building circuits on. It allows you to place components and connections on the board to make circuits without soldering. The holes in the breadboard take care of your connections by physically holding onto parts or wires where you put them and electrically connecting them inside the board. The ease of use and speed are great for learning and quick prototyping of simple circuits.

Think of the power rails as your canvas when painting—they run continuously along the length of the board, making it easy to distribute power across different parts of the circuit. Additionally, each column of five holes is internally connected, so any component legs placed in the same row will be electrically linked.

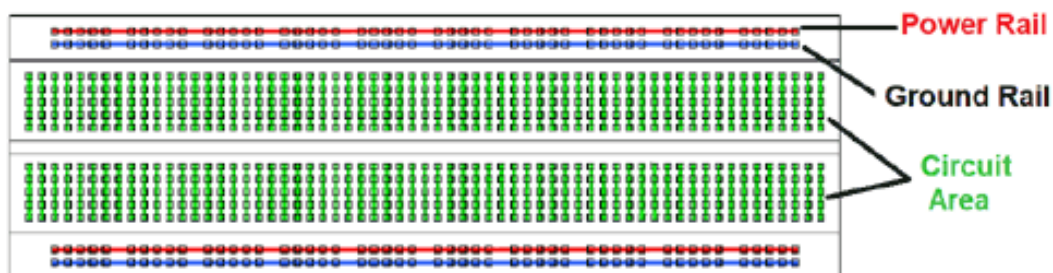


Figure 2: Illustrating the wiring connections of a breadboard

Jumper wires are small, insulated wires with connector pins at both ends, used to create temporary electrical connections in circuit prototyping. They are commonly used with breadboards and microcontrollers like Arduino to connect different components without the need for soldering.

Jumper wires come in various colors to help differentiate signals, with red typically indicating positive voltage and black representing ground. They are essential for testing and debugging circuits quickly and efficiently in electronics projects.

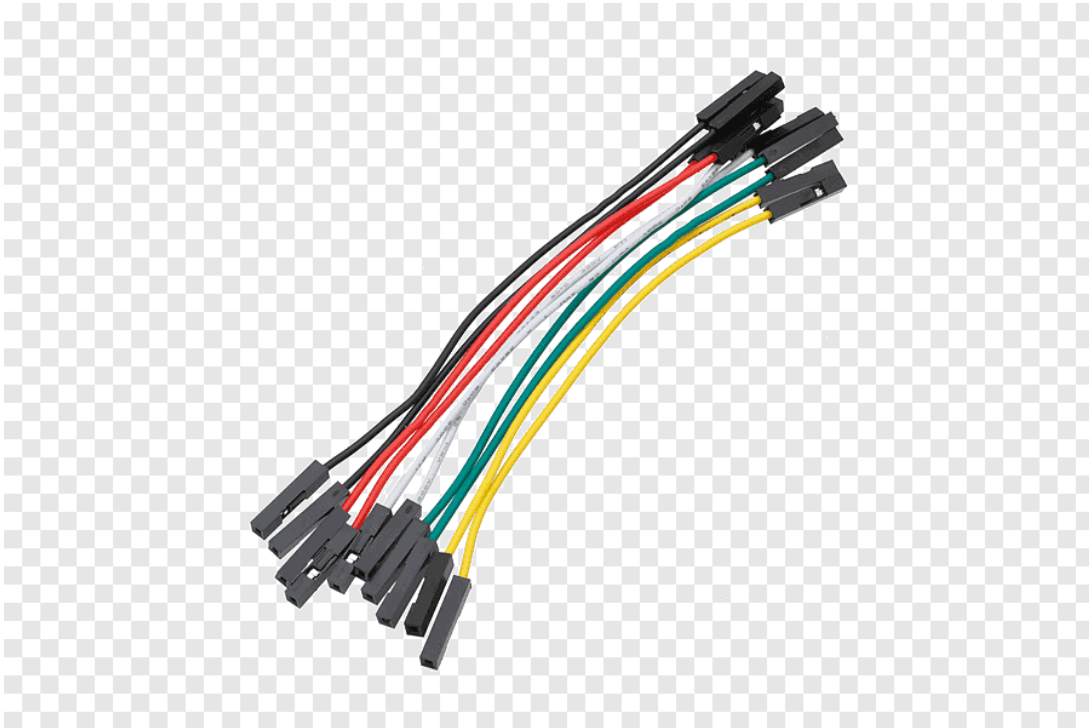


Figure 3: *Illustrating the wiring connections of a breadboard*

2.4 Components you will need for this lab

1. LED (Light Emitting Diode)
2. BreadBoard)
3. Pushbutton
4. Jumper Wires
5. 1 220 Ω resistor

2.5 Lab 0.1: Light up an LED without code

For our first lab, we will light up an LED without actually using the Arduino's microcontroller to send commands to its digital I/O pins. Instead, we will utilize the Arduino as a simple power source, providing either 5V or 3.3V relative to Ground. These voltage levels are sufficient to power basic components like LEDs. The designated power and ground pins are located on the left side of the Arduino board and are clearly marked for easy identification. By directly connecting our circuit to these pins, we can observe how an LED lights up using the provided voltage How to build the circuit:

1. First make sure that the Arduino is powered off (no USB cable plugged to anything).
2. Check the LED, you will see that one of the leg is shorter than the other one.
3. Plug the shorter leg of the LED to a hole on the breadboard. Connect that leg to a GND pin of the Arduino, using a black cable if possible (convention for GND).
4. Plug the longer leg of the LED to a different hole, on a different and independent line of the breadboard.
5. Add a 220 Ohm resistor between this longer leg and a digital pin of the Arduino, using an additional colored wire (no red, no black) for convenience.

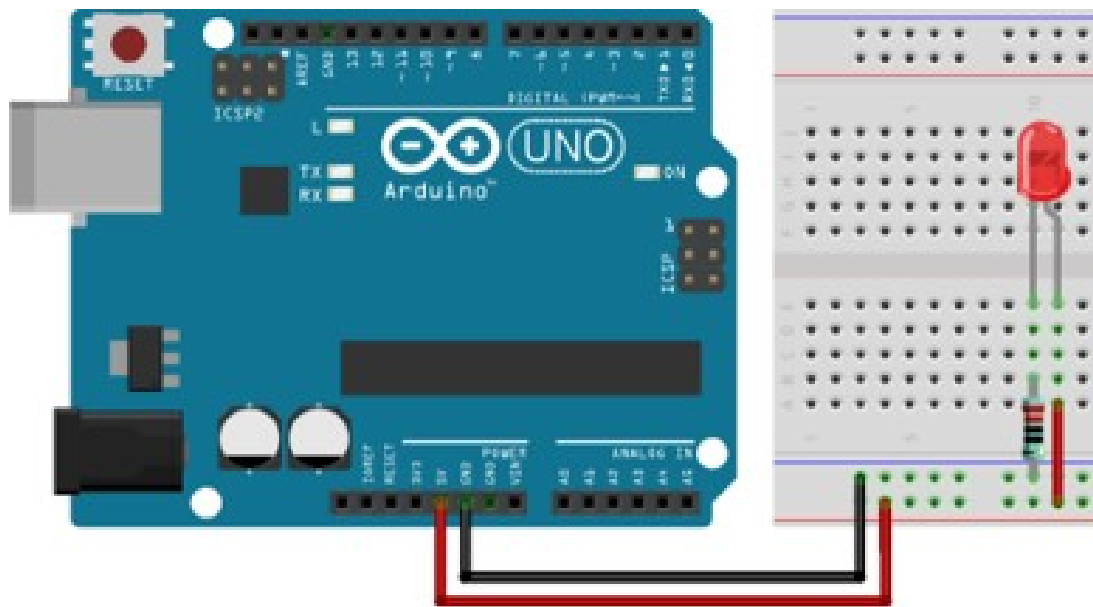


Figure 4: Wiring diagram for Lab 0.1

2.6 Lab 0.2: Light up an LED with code

Light your Led using code

How to build the circuit:

1. First make sure that the Arduino is powered off (no USB cable plugged to anything).
2. Check the LED, you will see that one of the leg is shorter than the other one.
3. Plug the shorter leg of the LED to a hole on the breadboard. Connect that leg to a GND pin of the Arduino, using a black cable if possible (convention for GND).
4. Plug the longer leg of the LED to a different hole, on a different and independent line of the breadboard.

5. Add a 220 Ohm resistor between this longer leg and a digital pin of the Arduino, using an additional colored wire (no red, no black) for convenience.

Note: you could have chosen any of the digital pins ranging from 0-13, and also any of the analog pins, which you can use as digital pins. So, for the Arduino Uno, you get 20 possibilities. More info on Arduino Uno pins.

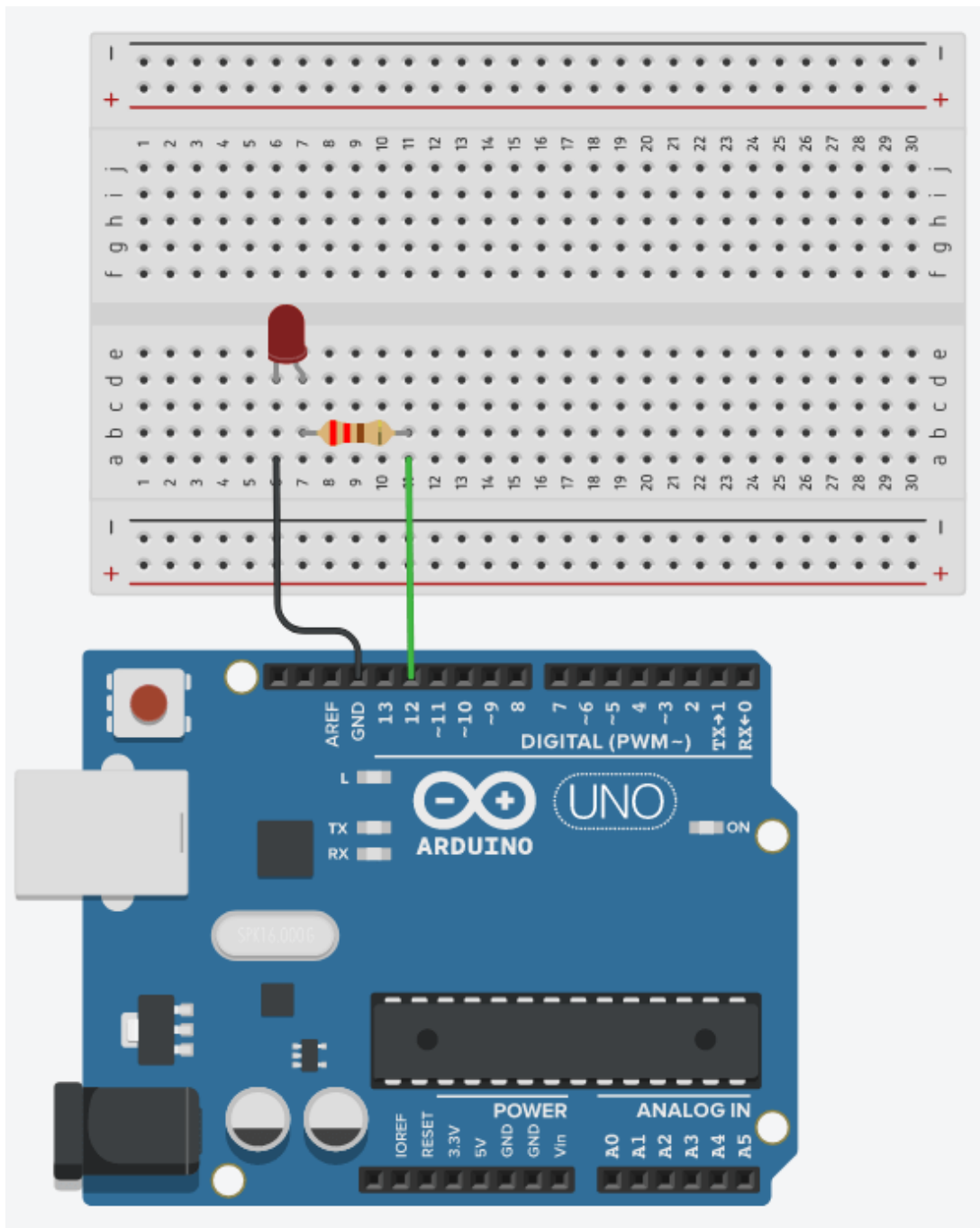


Figure 5: Wiring diagram for Lab 0.2

Once your are done with wiring copy paste the code below and upload

```
1  /*
2   Blink
3
4   Turns an LED on and off repeatedly.
5
6   The LED must be connected ot pin 12.
7  */
8
9  // the setup function runs once when you press reset or power the board
10 void setup() {
11     // initialize digital pin 12 as an output.
12     pinMode(12, OUTPUT);
13 }
14
15 // the loop function runs over and over again forever
16 void loop() {
17     digitalWrite(12, HIGH); // turn the LED on (HIGH is the voltage level)
18     delay(1500);           // wait for a second and a half
19     digitalWrite(12, LOW);  // turn the LED off by making the voltage LOW
20     delay(500);            // wait for half a second
21 }
```

2.7 Lab 0.3: Turn on an LED using a pushbutton

Finally, we will control an LED using a pushbutton. A push button is a simple switch that completes or interrupts a circuit when pressed. It allows current to flow only when the button is pressed, making it useful for momentary control of electrical components.

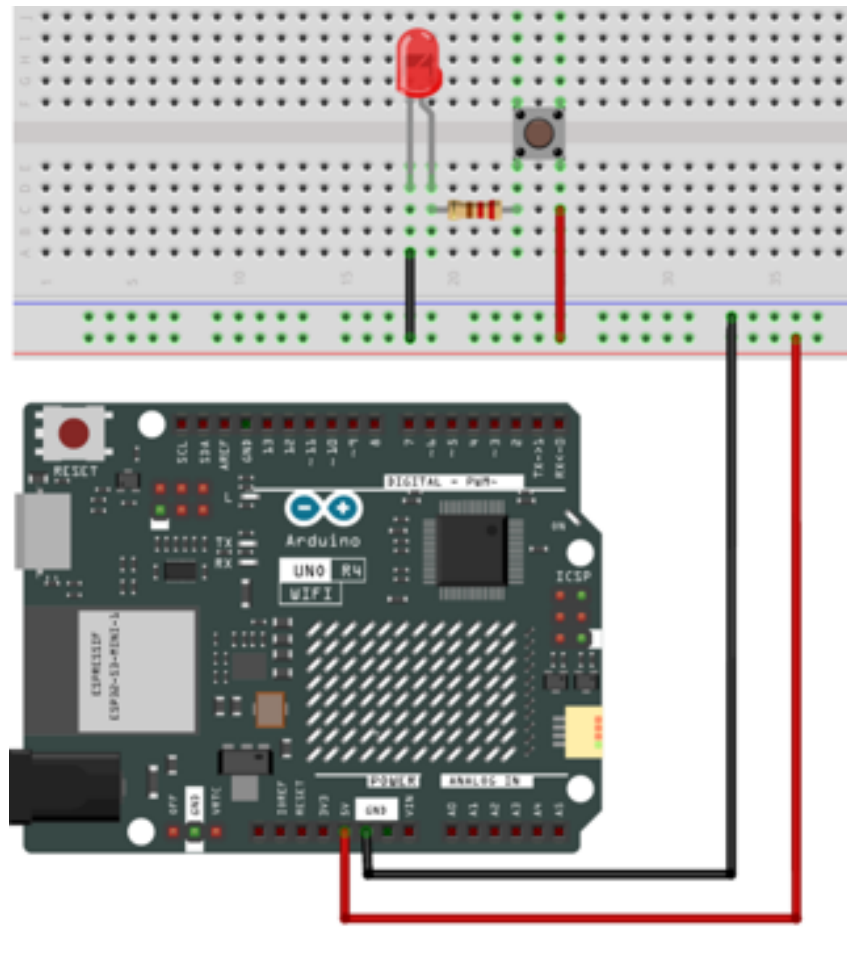


Figure 6: Wiring diagram for Lab 0.3

2.8 Lab 0 deliverables

Add a folder named **Lab0** to your **name_ESE1900** folder.

- **[Lab 0.1]** Photo of your circuit with the LED lit up
File name: lab0_1
- **[Lab 0.2]** Video of your circuit with the LED blinking
File name: lab0_2
- **[Lab 0.3]** Video of your button circuit lighting the LED
File name: lab0_3
- **[Lab 0.P]** Screenshot of #general in the Slack channel
File name: lab0_P

Add the share link to your 'Lab0' folder in the post lab quiz Set access to anyone with link can view

Post-lab 0 submission due 5:15 PM 1/28/2025

3 Lab 2: Sensing

In this lab we will finally get our hands on our first sensors. Sensors are crucial components in many devices, as they allow us to measure physical properties such as temperature, light, distance, and motion, converting them into electrical signals that a microcontroller can process. By working with sensors in this lab, we will lay the groundwork for more complex projects.

3.1 Lets start with some theory first

It is crucial to understand how the Arduino can read the values coming from a sensor. When we interface sensors with the microcontroller, the output of the sensor many times is analog in nature. However, the microcontroller processes digital signals. Hence, we use an ADC in the middle between the sensor and the microcontroller. It converts an analog signal into a digital signal and gives it to the microcontroller.

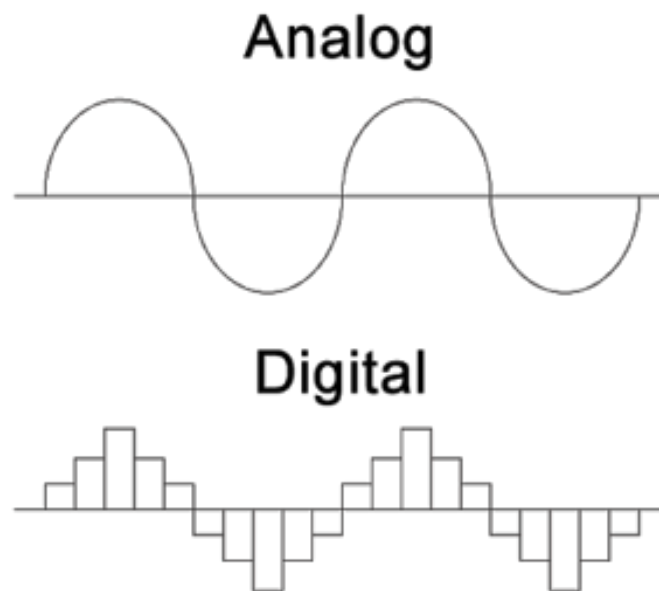


Figure 7: *Analog vs Digital signal*

3.2 Lab 2.1: Ping Sensor Serial Monitor readings

First sensor we will be implementing to our lab is the ping sensor. The ping sensor works by sending out a burst of ultrasound and listening for the echo when it bounces off of an object.

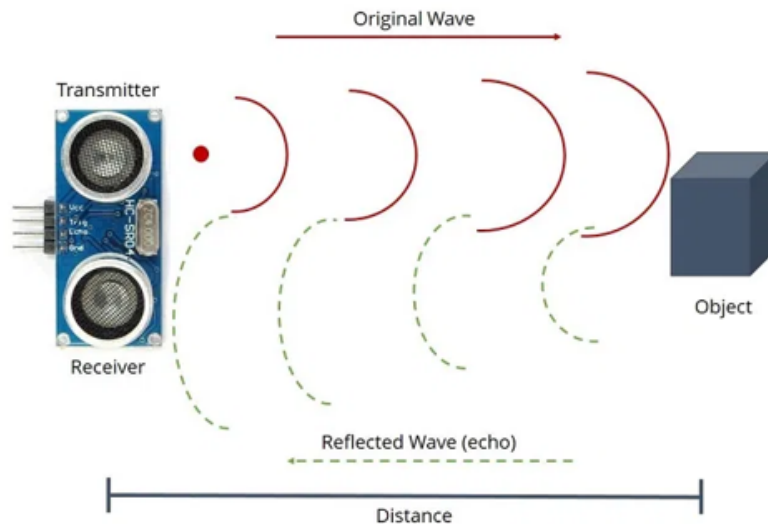


Figure 8: *Analog vs Digital signal*

Did You Know?

Bats locate their prey by producing sound waves and listening for an echo back. This closely resembles the working principle of a ping sensor!

But how does the ping sensor communicate the measured distance to the arduino. Just like us humans sensors have a method communicating with one another. This is better illustrated in figure 9:

- Ping sensor sends a signal (trig) which lets the arduino know that its about to send some form of data
- Next we got 8 sonic bursts. This 8-pulse pattern is specially designed so that the receiver can distinguish the transmitted pulses from ambient ultrasonic noise and not read false data. Cool right?
- Finally we got the actual digital signal whose duration represents the measured distance. The arduino then measures the length of that signal and converts it to a readable value.

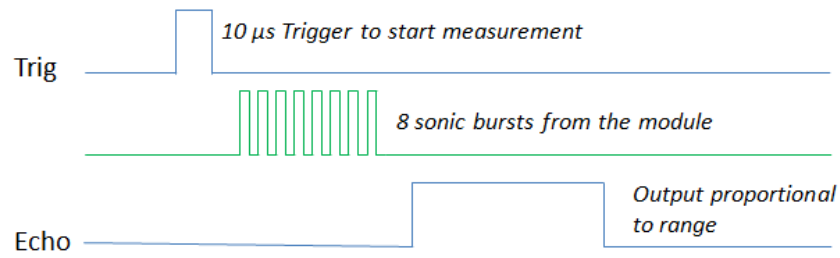
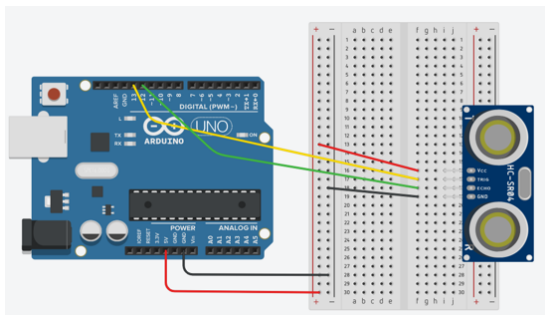


Figure 9

3.2.1 How to wire the ping sensor to the arduino.

Simple stuff. You got this!



Arduino	Ping Sensor
+5V	VCC
GND	GND
PIN 13	TRIG
PIN 12	ECHO

Figure 10: Wiring for 2.1

3.2.2 Code for Lab 2.1

```
1  /*
2
3  ESE 1900
4  Fall 2024
5  Lab 2: Analog and Digital Sensing Part 1
6  2.2a Ping Sensor
7
8  Code Credit: Arduino
9  Modified by James Steeman
10
11 */
12
13 // declare pins
14 #define trigPin 13
15 #define echoPin 12
16
17 void setup () {
18     // Initialize serial
19     Serial.begin (9600);
20     delay(500);
21
22     // define pin modes
23     pinMode (trigPin, OUTPUT);
24     pinMode (echoPin, INPUT);
25 }
26
27 void loop () {
28     long duration , distance;
29     digitalWrite (trigPin , LOW ); // start trig at 0
30     delayMicroseconds (2);
31     digitalWrite (trigPin , HIGH ); //The rising edge of trig pulse
32     delayMicroseconds (10); // decides duration of trig pulse
33     digitalWrite (trigPin , LOW ); //falling edge of the trig pulse
34     // NOTE: echo pin reads HIGH till it receives the reflected signal
35     duration = pulseIn (echoPin , HIGH ); // Reading the duration for which echoPin was HIGH
36     //you the time the sensor receives a reflected signal at the echo pin
37     distance = (duration / 2) / 29.1; //Calculate the distance of the reflecting surface in cm
38
39     Serial.print("distance:");
40     Serial.println(distance);
41     delay(500);
42 }
```


3.3 Lab 2.2: Ping Sensor controlling RGB LED

How about we do something useful with our sensor readings? In this deliverable, you will add an RGB LED that changes color based on the distance measured by the ultrasonic sensor. Our new addition is the RGB LED. This differs from the LEDs used in Lab 0 because it can emit red, green, and blue light. It has three separate pins for controlling the red, green, and blue LEDs inside.

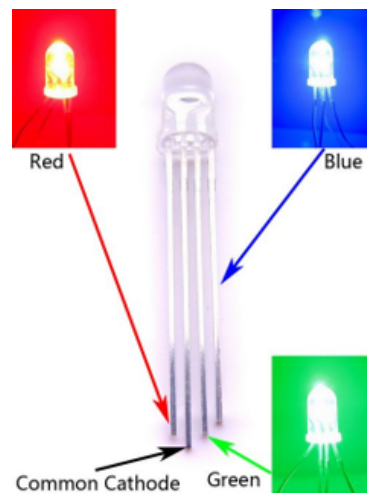


Figure 11: RGB led

3.3.1 Wiring is simple.

Remember to use 330 Ω valued resistors!

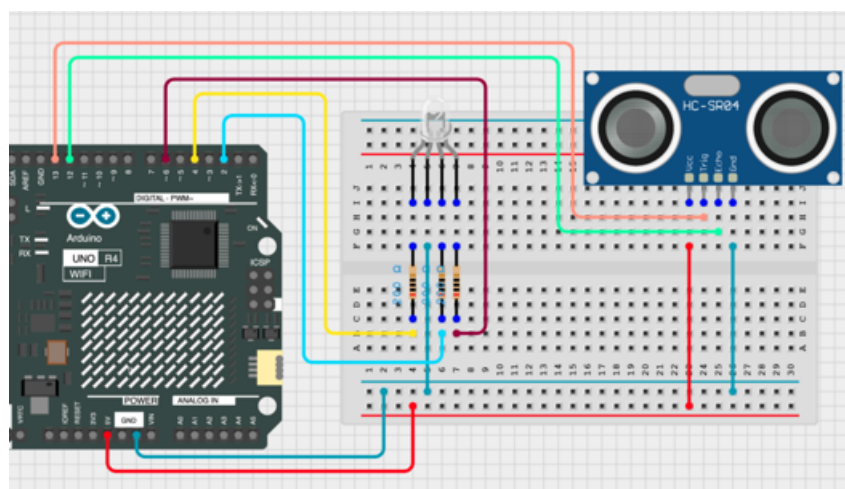


Figure 12: Wiring diagram for Lab 2.2

3.3.2 Code for Lab 2.2

The 2.2 code is essentially the same code as the 2.1 code for reading ping sensor data, however there is an additional segment dedicated to controlling the LED.

For this circuit, we are going to turn on a different color of the RGB LED depending on the distance read by the ping sensor.

Follow the comments to see which LED should turn on in each case and implement logic to ensure the proper LED is on

Distances are in centimetres and you should set the threshold values yourself

```
1  /*
2
3  ESE 1110
4  Fall 2024
5  Lab 2: Analog and Digital Sensing Part 1
6  2.2b Ping Sensor and RGB LED
7
8  Code Credit: Arduino
9  Modified by Oscar Qu
10
11 */
12
13 // declare pins
14 const int trigPin = 13;
15 const int echoPin = 12;
16 const int LED_red = 4;
17 const int LED_green = 2;
18 const int LED_blue = 6;
19
20 void setup() {
21     // Initialize serial and wait for port to open:
22     Serial.begin(9600);
23     // This delay gives the chance to wait for a Serial Monitor without blocking if none is
        found
24     delay(1500);
25
26
27     // define pin modes
28     pinMode(trigPin, OUTPUT);
29     pinMode(echoPin, INPUT);
30     pinMode(LED_red, OUTPUT);
31     pinMode(LED_green, OUTPUT);
32     pinMode(LED_blue, OUTPUT);
33 }
34
35 void loop() {
36     long duration, distance;
```

```

37 digitalWrite(trigPin, LOW); // start trig at 0
38 delayMicroseconds(2);
39 digitalWrite(trigPin, HIGH); // The rising edge of trig pulse
40 delayMicroseconds(10); // decides duration of trig pulse
41 digitalWrite(trigPin, LOW); // falling edge of the trig pulse
42 // NOTE: echo pin reads HIGH till it receives the reflected signal
43 duration = pulseIn(echoPin, HIGH); // Reading the duration for which echoPin was HIGH gives
44 // you the time the sensor receives a reflected signal at the echo pin
45 distance = (duration / 2) / 29.1; // Calculate the distance of the reflecting surface in cm
46
47 Serial.print("distance:");
48 Serial.println(distance);
49
50
51 if (/*FILL THIS IN */) // Below minimal threshold distance turn on Red LED
52 {
53     digitalWrite(LED_red, /*FILL THIS IN*/); // HIGH or LOW
54     digitalWrite(LED_green, /*FILL THIS IN*/); // HIGH or LOW
55     digitalWrite(LED_blue, /*FILL THIS IN*/); // HIGH or LOW
56 }
57 else if (/*FILL THIS IN*/) // Below another threshold run on Green LED
58 {
59     digitalWrite(LED_red, /*FILL THIS IN*/); // HIGH or LOW
60     digitalWrite(LED_green, /*FILL THIS IN*/); // HIGH or LOW
61     digitalWrite(LED_blue, /*FILL THIS IN*/); // HIGH or LOW
62 }
63 else // if the distance is larger than both thresholds, turn on Blue LED
64 {
65     digitalWrite(LED_red, /*FILL THIS IN*/); // HIGH or LOW
66     digitalWrite(LED_green, /*FILL THIS IN*/); // HIGH or LOW
67     digitalWrite(LED_blue, /*FILL THIS IN*/); // HIGH or LOW
68 }
69
70 delay(500);
71
72 }

```

3.4 Lab 2.3: Temp/Humidity sensor readings in Serial monitor

Lets switch gears and focus on another familiar sensor. It is called a dht11

3.5 Lab 2 deliverables

Add a folder named **Lab2** to your **name_ESE1900** folder.

- **[Lab 2.1]** Video of your Ping Sensor Serial Monitor readings
File name: lab2_1
- **[Lab 2.2]** Video of your Ping Sensor controlling RGB LED
File name: lab2_2
- **[Lab 2.3]** Video of your Temp/Humidity sensor readings in Serial monitor
File name: lab2_3

References